PIRTM is the other L0. It does not compile a Foundry model, and it is not a stage inside PrismPM. It is the contractive kernel on the PhaseMirror side: a session is admitted only if its reflection cannot expand. The workshop assumes that law. It does not yet call it.

What the platform is

PIRTM/MOC is Phase Mirror / Multiplicity Object Code, governed platform v1.1.0. The repository describes it as a formally verified, contractive L0 development platform, compiler, and interactive terminal. The legal entity is Citizen Gardens UNA, a Wyoming unincorporated nonprofit association under W.S. 17-22, doing business as The Prime Materia Commons. Licenses are the Prime Materia Open Commons License v1.0 and the Lawful Recursion License (Ξ-License v1.0).

The kernel it integrates is named, not implied: Sedona Spine, prime-indexed tensor sheaf contractions, and Lyapunov drift bounding. Source is `.pirtm`. The compiler lowers it through MLIR toward LLVM. The proof core is Lean 4 with no Mathlib. The README does not mention Foundry, PrismPM, or Hologram. That absence is part of the joint. Coupling is by law, not by import.

A program session executes only after two receipts:

- transpile-time receipts;
- link-time spectral small-gain contractivity, anchored to machine-checked Lean 4 declarations.

The gate is

$$\|G\|_1 = \max_j \sum_i |A_{ij}| \cdot \lambda_j < 1 \quad \text{in } \mathbb{Q}$$

held over exact reduced rational tuples $\mathrm{PosRat} \in \mathbb{Q}$. `spectral.rs` evaluates it. ADR-055 removed a 10^6 float scaling membrane and replaced it with `Ensemble::from_rationals` over reduced `PosRat`. A float that rounds under 1 is not a pass. An expansive edit is not infrastructure.

The substrate, as built

```text
```

```
.pirtm
   |
   v
pirtm-compiler ── MLIR lowering
   |
   v
Lean 4 proof core (no Mathlib) ── Sentinel ── ||G||_1 < 1 in Q
   |
   v
pirtmd  (ws://127.0.0.1:8090, JSON-RPC 2.0, WardMonitor + MCP)
   |
   v
TUI / VS Code / Neovim
```

The status table in v1.1.0 separates verified from implemented:

Subsystem	Status	Anchor
Lean 4 proof core	Verified	25 ADR modules, zero `sorry`
Small-gain gate	Verified	`spectral.rs` , ADR-055
Rust workspace	Verified	24 crates
Kani harnesses	Verified	`adr_rust`
Poseidon2 receipts	Verified	ADR-049
WardMonitor / Lyapunov	Verified	ADR-048
Multi-node quorum	Verified	ADR-050
CRDT merge	Verified	ADR-056, contractivity preserved
`pirtmd` , TUI, editor IPC	Implemented	daemon and clients, not a second proof

ADR-034 through ADR-056 are the accepted series inside this repository. They are not the Foundry ADR numbers. ADR-048 is the Lyapunov tripwire: WardMonitor may halt a drift before the session treats it as stable. ADR-052 states a prime-valuation homomorphism and weighted-ℓ_1 soft-thresholding as non-expansive. ADR-056 is the collaborative clause: a CRDT vector-clock merge must converge and must preserve $\|G\|_1 < 1$. Merged civic edits are still subject to the gate. Consensus does not waive it.

Sedona Spine, on the Foundry ledger, is a separate accepted decision: ADR-002 names the Sedona Spine retention engine the sole source of truth. ADR-008 makes the prime-

signature canonical monoid the exclusive Rust kernel substrate. ADR-009 requires a contraction witness and spectral-radius verification before the emission gate. ADR-010 is the axiom-clean kernel boundary and the manifested proof-debt policy — the same honesty family as `alp_sorry_manifest.json`. Those Foundry ADRs are the civic statement of the law. PIRTM v1.1.0 is the governed language that claims to enforce a form of it. They are not one repository.

What contractive means here

Phase mirror is not “generate more views.” A view, a guest, a CRDT merge, or a lowered module is a reflection. Contractive means the reflection is a map whose gain is strictly less than one, so iterated reflection converges instead of amplifying drift.

In the PIRTM gate the map is a matrix A with weights λ_j, and the gain is the column-sum norm above. If $\|G\|_1 \geq 1$, the Sentinel does not open the session. There is no degraded mode in the execution condition the README states. The Foundry attestation boundary is the civic counterpart, and it lives in a different record: ADR-0110, OSCAL and PrismPM, gated in PhaseMirror/Foundry. The report dated 2026-09-17 is PASS on index, sorry-check, `lake build ADR`, and `lake test`. The model the gate exercises is Scopist determinism and idempotency, zero-drift — `SIG_GOV_KILL ⇒ UNATTESTED` — an attestation boundary, packaging-only OSCAL, and registry invariants: unique ids, acyclicity, supersession, traceability, immutability. A governance kill does not leave a half-attested object. It leaves an unattested one. OSCAL in that model packages evidence. It does not become a second source of behavior.

The ADR-0110 source file is not at the path the report names; the passing report is what the repository publishes. PIRTM is not cited in that gate. The implication for the workshop is methodological: PrismPM may seal a Hologram, and PIRTM may refuse a session, and both refusals are the same kind of act. Drift that fails the small-gain gate, or a kill that fails to drop attestation, does not become infrastructure.

What this layer does not do

PIRTM does not author `.lex.tex`, does not emit `model.prism.json`, and does not seal a `.holo`. LexLean’s closures are lexical and axiomatic. PrismPM’s closures are viewpoint and artifact identity. Hologram’s closures are archive kappa and a foreign oracle. PIRTM’s closure is spectral: the lowered system is non-expansive in exact rationals, or it does not run.

That is why it sits under the Foundry rather than inside the compiler pipeline. The civic R&D workshop can mirror a site, a control, or a workflow into proof, package, and archive. The

base-state still assumes that the mirror itself is contractive — that a feedback edit, a merged workshop state, or a governance signal cannot gain. Until a Foundry model is actually lowered through this gate, that assumption is a law the platform states, not a receipt the portal holds. The receipt, on the PIRTM side, is the Sentinel pass. On the Foundry side, it is an attestation that a kill can still void.